

Practice Exercises for Decorators

Basic Decorator – Logging

Create a decorator named `log_call` that prints:

```
Calling function <function_name>
```

before the function runs.

Task:

Decorate three functions: `add(a, b)`, `greet(name)`, and `square(n)`.

Decorator That Measures Execution Time

Create a decorator called `measure_time` that prints how many seconds a function took to execute.

Task:

Decorate a function `slow_task()` that sleeps for 2 seconds.

Decorator with Arguments (like `role_required`)

Write a decorator named `repeat(n)` that runs the function **n times**.

Example:

```
@repeat(3)
def hello():
    print("Hello!")
```

Output:

```
Hello!  
Hello!  
Hello!
```

Create Your Own Cache Decorator

Write a decorator `memoize` that:

- stores results in a dictionary
- prints `"Fetching from cache"` when the result already exists
- prints `"Calculating..."` when running the original function

Task:

Decorate a function `fib(n)` that returns the nth Fibonacci number.

login_required Practice

Given:

```
logged_in = False
```

Create:

- `profile()` → only logged-in users can see
- `settings()` → only logged-in users can access

Decorate both with `@login_required`.

Try toggling:

```
logged_in = True
```

and test again.

role_required(["admin", "editor"])

Create a system with:

```
role = "viewer"
```

Make functions:

- `delete_post()`
- `edit_post()`
- `view_stats()`

Use:

```
@role_required(["admin"])
@role_required(["admin", "editor"])
@role_required(["admin", "editor", "viewer"])
```

Test by changing `role` values.

Multiple Decorators Together

Write two decorators:

- `uppercase` → converts output text to uppercase
- `exclaim` → adds `"!!!"` at the end

Decorate:

```
@exclaim
@uppercase
def greet():
    return "good morning"
```

Expected:

```
GOOD MORNING!!!
```

Then switch the decorator order and observe the difference.

Decorator That Validates Input

Write a decorator `ensure_positive` that checks if all arguments are positive.

Example:

```
@ensure_positive
def divide(a, b):
    return a / b
```

If user calls:

```
divide(-4, 2)
```

Output:

```
All numbers must be positive
```

Decorator That Counts How Many Times a Function Was Called

Create `count_calls` decorator.

It should print:

```
Function <name> has been called <n> times
```

Task: Apply it to any two functions.
